

Lecture 5 – The gradient descent algorithm

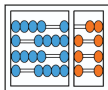
[Partly based on Andrew Ng's (Stanford) lectures]

MC886 – Machine Learning

Marcelo S. Reis ^{1,2}

¹ Departamento de Sistemas de Informação
Instituto de Computação, Universidade Estadual de Campinas

Campinas, March 9, 2026



**Instituto de
Computação**

UNIVERSIDADE ESTADUAL DE CAMPINAS



UNICAMP

Today's summary

Closed-form solution for multiple linear regression

Optimization problem formulation

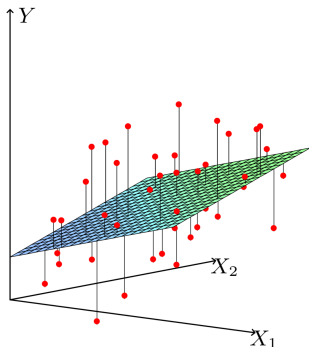
The gradient descent (GD) algorithm

Improving the GD performance

- ❖ Often we want to predict a linear association between Y and two or more independent variables $X_1, \dots, X_p$

$$\begin{aligned} Y &\approx \hat{\beta}_0 + \hat{\beta}_1 X_1 + \dots + \hat{\beta}_p X_p \\ &= \hat{\beta}_0 + \sum_{j=1}^p \hat{\beta}_j X_j \\ &= \sum_{j=0}^p \hat{\beta}_j X_j && (X_0 = 1) \\ &= \hat{\beta}^T X \end{aligned}$$

Multiple linear regression ($p = 2$)



$$\begin{aligned}\hat{y}_i &\approx \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2} \\ &= \hat{\beta}_0 \underbrace{x_{i0}}_1 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2} \\ &= \hat{\beta}^T x_i\end{aligned}$$

Mean-squared error (MSE)

$$\begin{aligned}MSE(\hat{\beta}) &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\&= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2 \quad \left(x_i = (x_{i0}, \dots, x_{ip})^T\right) \\&= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\beta}^T x_i)^2 \\&= \frac{1}{n} (\mathbf{y} - \mathbf{X}\hat{\beta})^T (\mathbf{y} - \mathbf{X}\hat{\beta})\end{aligned}$$

where:

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \quad \text{and} \quad \mathbf{X} = \begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix}$$

Before minimize our *MSE*: some review

- ❖ Consider a function with two or more variables

$$\text{Example: } f(x_1, x_2) = 7x_1^2 + 4x_1x_2 - 666x_2 + 333$$

We can assess the sensitivity to change of f evaluating one argument at time and fixing the others as constants:

$$\frac{\partial f}{\partial x_1}(x_1, x_2) = 14x_1 + 4x_2 - 0 + 0 = 14x_1 + 4x_2$$

$$\frac{\partial f}{\partial x_2}(x_1, x_2) = 0 + 4x_1 - 666 + 0 = 4x_1 - 666$$

Gradient

The **gradient** of a scalar-valued, differentiable function $f : \mathbb{R}^p \rightarrow \mathbb{R}$ is defined as:

$$\nabla f(x_1, \dots, x_p) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x_1, \dots, x_p) \\ \vdots \\ \frac{\partial f}{\partial x_p}(x_1, \dots, x_p) \end{pmatrix}$$

Example of gradient

$$f(x_1, x_2) = 7x_1^2 + 4x_1x_2 - 666x_2 + 333$$

$$\nabla f(x_1, x_2) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x_1, x_2) \\ \frac{\partial f}{\partial x_2}(x_1, x_2) \end{pmatrix} = \begin{pmatrix} 14x_1 + 4x_2 \\ 4x_1 - 666 \end{pmatrix}$$

Minimizing MSE

$$\begin{aligned} n \cdot MSE(\hat{\beta}) &= \frac{1}{n} (\mathbf{y} - \mathbf{X}\hat{\beta})^T (\mathbf{y} - \mathbf{X}\hat{\beta}) \\ &= RSS(\hat{\beta}) \end{aligned}$$

Differentiating $RSS(\hat{\beta})$ with respect to $\hat{\beta}$ we obtain:

$$\nabla RSS(\hat{\beta}) = -2\mathbf{X}^T (\mathbf{y} - \mathbf{X}\hat{\beta})$$

Assuming $\mathbf{X}^T \mathbf{X}$ is invertible, we can set the above gradient to zero and, after some matrix algebra, obtain:

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

The Linear Regression Algorithm (LRA)

$$\text{LRA}(\{(x_1, y_1), \dots, (x_n, y_n)\})$$

$$1 \quad \mathbf{X} \leftarrow \begin{pmatrix} x_1^T \\ \vdots \\ x_n^T \end{pmatrix}$$

$$2 \quad \mathbf{y} \leftarrow \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

$$3 \quad \mathbf{X}^+ \leftarrow (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \quad \triangleright \text{Moore-Penrose pseudoinverse of } \mathbf{X}$$

$$4 \quad \text{return } \mathbf{X}^+ \mathbf{y} \quad \triangleright \hat{\beta} = \mathbf{X}^+ \mathbf{y}$$

Computational issues for this closed-form solution

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Caveat: to compute $\mathbf{X}^T \mathbf{X}$ is $O(n^3)$

$\Rightarrow$ consider $p = O(n)$

n	n^3
100	1,000,000
1000	1,000,000,000
10000	1,000,000,000,000

Minimizing MSE as an optimization problem

Function: $Y = \beta_0 + \beta_1 X$

Parameters: $\beta = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix}$

Cost function: $J(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2$

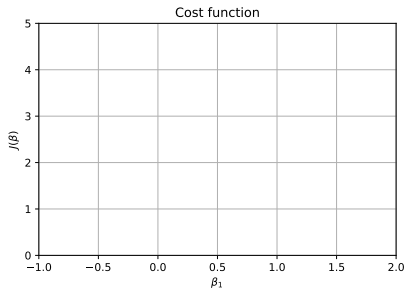
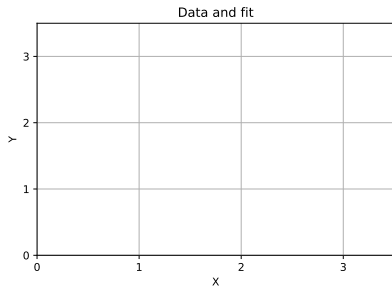
Objective: minimize $J(\beta)$
 β

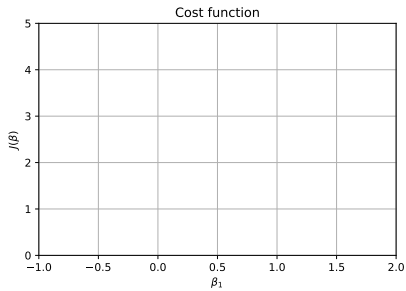
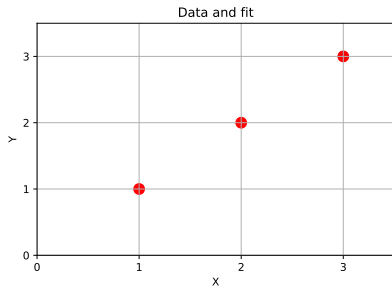
$$\begin{aligned}\underset{\beta}{\text{minimize}} \quad J(\beta) &= \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2 \\ &= \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2\end{aligned}$$

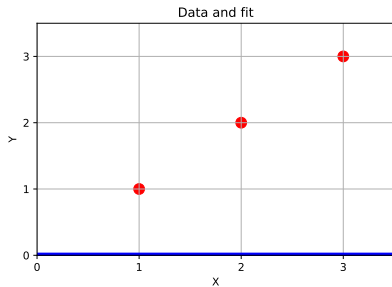
How changes in β affects $f(x_i)$?

Let's visualize it graphically

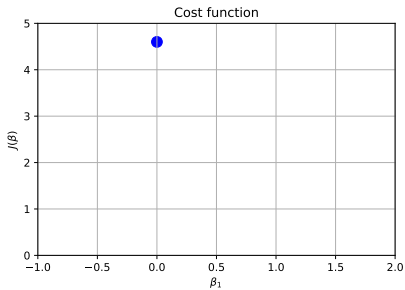
- For the sake of simplicity, we'll set $\beta_0 = 0$



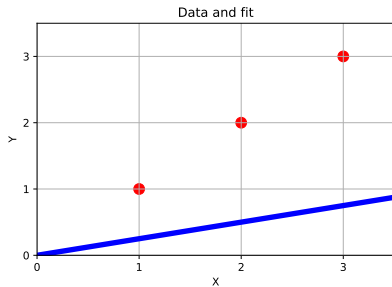




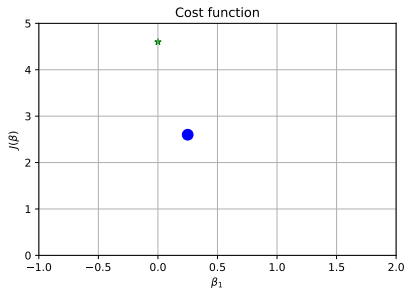
$$\hat{\beta} = \begin{pmatrix} \beta_0 = 0.00 \\ \beta_1 = 0.00 \end{pmatrix}$$



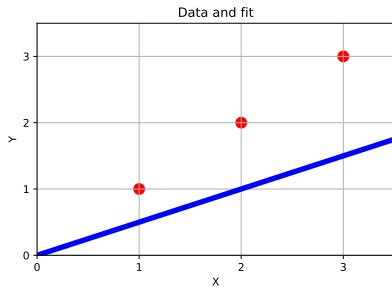
$$J(\hat{\beta}) \approx 4.6$$



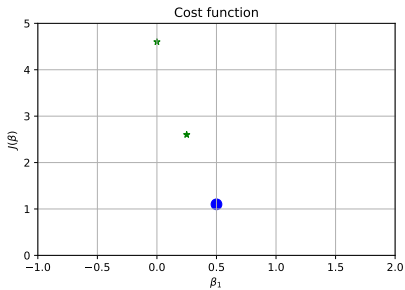
$$\hat{\beta} = \begin{pmatrix} \beta_0 = 0.00 \\ \beta_1 = 0.25 \end{pmatrix}$$



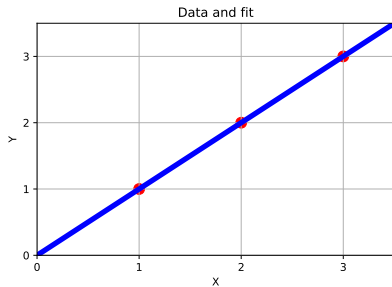
$$J(\hat{\beta}) \approx 2.6$$



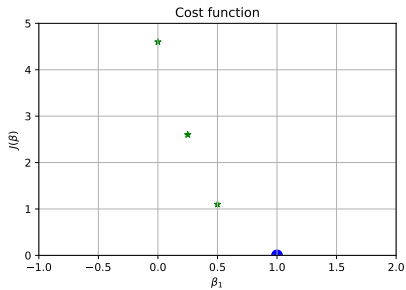
$$\hat{\beta} = \begin{pmatrix} \beta_0 = 0.00 \\ \beta_1 = 0.50 \end{pmatrix}$$



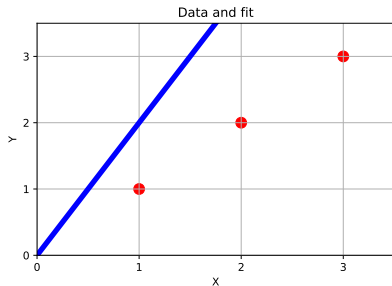
$$J(\hat{\beta}) \approx 1.1$$



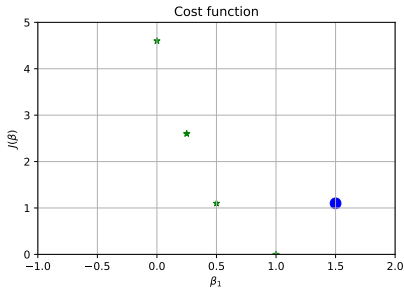
$$\hat{\beta} = \begin{pmatrix} \beta_0 = 0.00 \\ \beta_1 = 1.00 \end{pmatrix}$$



$$J(\hat{\beta}) = 0$$



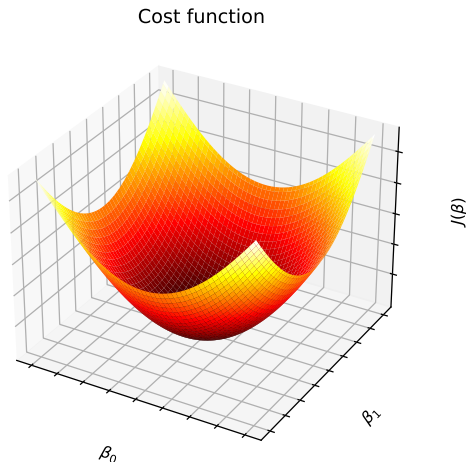
$$\hat{\beta} = \begin{pmatrix} \beta_0 = 0.00 \\ \beta_1 = 1.50 \end{pmatrix}$$



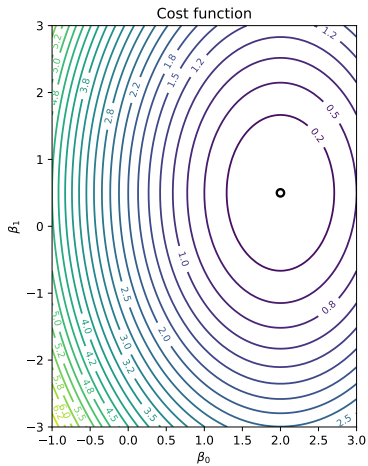
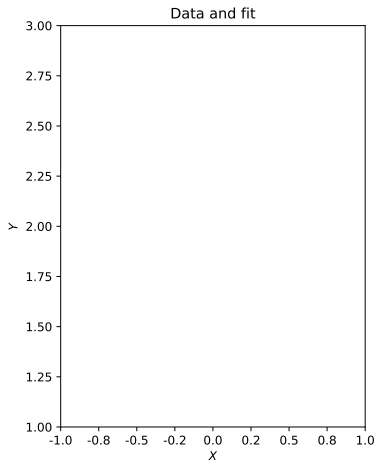
$$J(\hat{\beta}) \approx 1.1$$

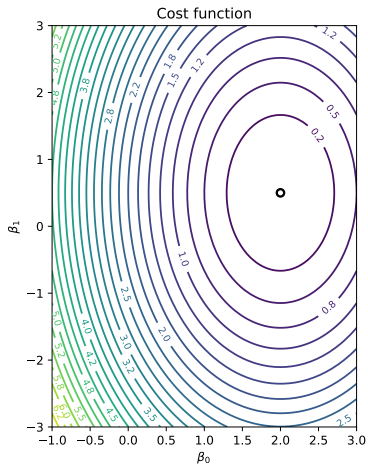
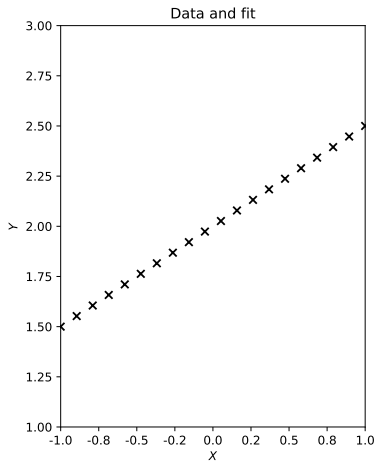
$J(\beta)$ seems to be a bell-shaped function

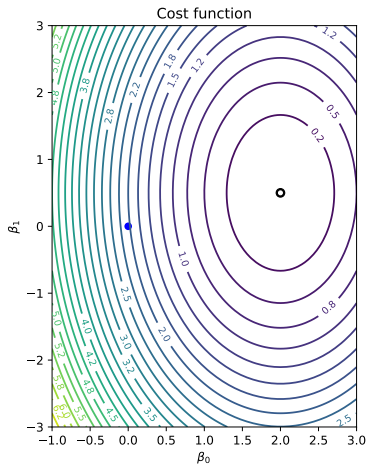
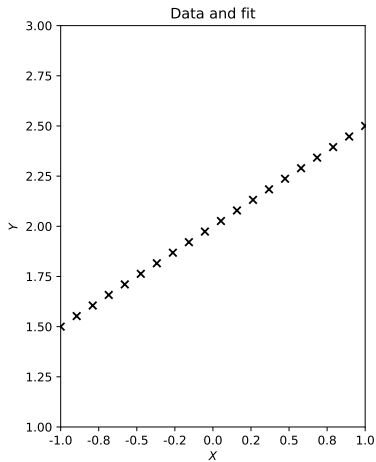
Visualizing two parameters at once affecting $J(\beta)$?



Even better: **Contour plots!**

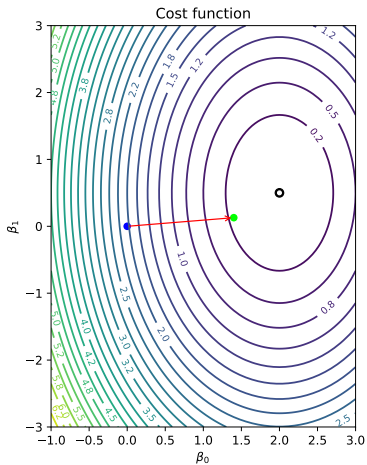
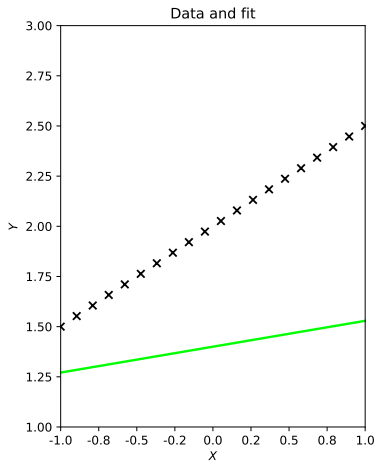






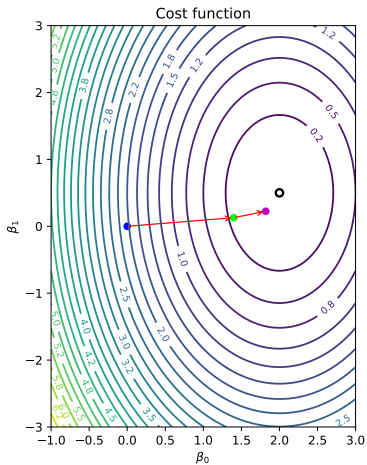
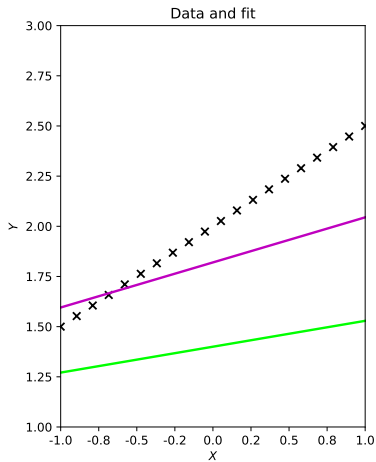
$$\hat{\beta} = \begin{pmatrix} \beta_0 = 0.00 \\ \beta_1 = 0.00 \end{pmatrix}$$

$$J(\hat{\beta}) \approx 2.046$$



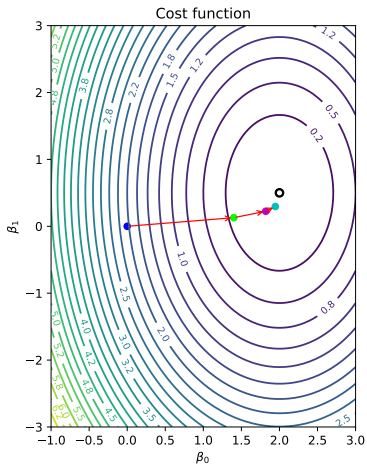
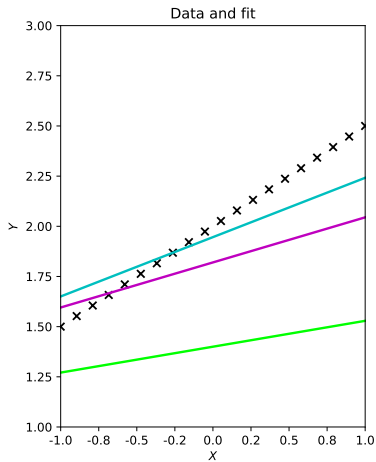
$$\hat{\beta} = \begin{pmatrix} \beta_0 \approx 1.40 \\ \beta_1 \approx 0.13 \end{pmatrix}$$

$$J(\hat{\beta}) \approx 0.205$$



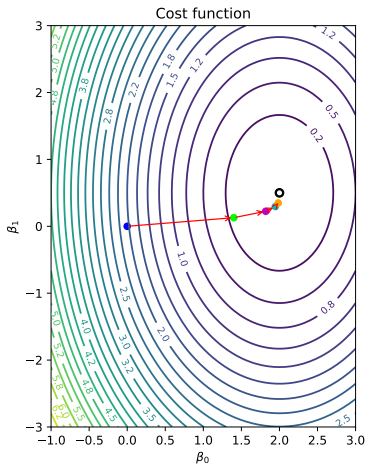
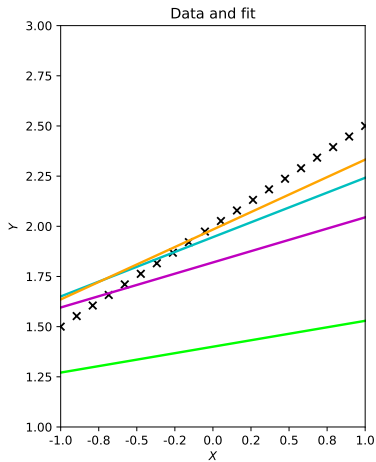
$$\hat{\beta} = \begin{pmatrix} \beta_0 \approx 1.82 \\ \beta_1 \approx 0.22 \end{pmatrix}$$

$$J(\hat{\beta}) \approx 0.030$$



$$\hat{\beta} = \begin{pmatrix} \beta_0 \approx 1.94 \\ \beta_1 \approx 0.13 \end{pmatrix}$$

$$J(\hat{\beta}) \approx 0.009$$



$$\hat{\beta} = \begin{pmatrix} \beta_0 \approx 1.98 \\ \beta_1 \approx 0.34 \end{pmatrix}$$

$$J(\hat{\beta}) \approx 0.004$$

The optimum is at the “bottom” of $J(\beta)$

How to know the direction to take at each step?

Idea: We can try to look where it goes down!

- Look at the gradient $\nabla J(\beta)$

Update recipe for β_1 :

$$\beta_j \leftarrow \beta_j - \alpha \frac{\partial}{\partial \beta_j} J(\beta)$$

Constant α is also known as *learning rate*

α Too small: can take a long time to converge

α **Too large:** can overshoot the minimum

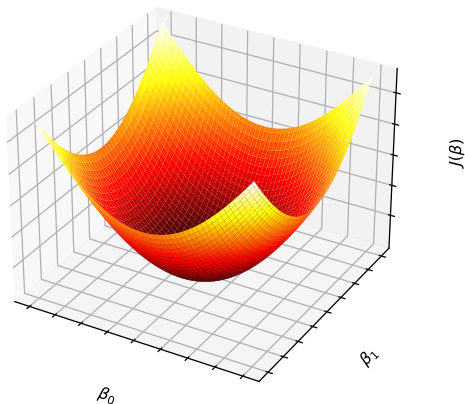
The gradient descent algorithm

GRADIENT-DESCENT($\{(x_1, y_1), \dots, (x_n, y_n)\}$)

```
1   $\beta \leftarrow \mathbf{0}$ 
2  while there is no convergence
3      do for each  $j$  in  $\{0, \dots, p\}$ 
4          do  $\beta'_j \leftarrow \beta_j - \alpha \frac{\partial}{\partial \beta_j} J(\beta)$ 
5           $\beta \leftarrow \beta'$   $\triangleright \beta' = \begin{pmatrix} \beta'_0 \\ \vdots \\ \beta'_p \end{pmatrix}$ 
6  return  $\beta$ 
```

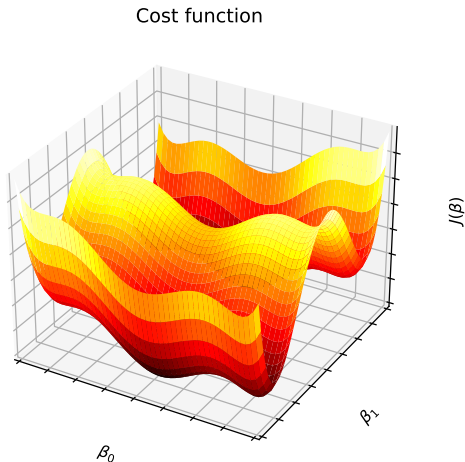
At each iteration, all data points are used (“batch” GD)

Cost function



MSE cost function is **convex** in linear regression!

Example of non-convex cost function



Improving the GD performance

1. Variable scaling
2. Choice of a suitable learning rate α

Variable scaling

If variables are in different scale, GD can be slow to converge

x_1 = house size (0-2000 m²)

x_2 = number of bedrooms (1-5)

Rule of thumb: put every feature in a $-1 \leq x_i \leq 1$ range

Mean normalization

Replace each x_{ij} in $x_{1j}, \dots, x_{nj}$ by:

$$\frac{x_{ij} - \bar{x}_j}{S_1}$$

S_1 can be:

- The maximum value
- The range (maximum - minimum)
- The standard deviation

Mean normalization does not apply to $x_{i0} = 1$

Adjusting the learning rate

- ❖ Plot $J(\beta)$ as the algorithm iterates
- ❖ Define a minimum step size as a stop criterion
- ❖ Tradeoffs: α size, correctness and performance